

Reconstrucción de imágenes incompletas a través de Deep Learning

Luis Adrián Uribe Cruz

Ingeniería en Robótica y Sistemas Digitales

Instituto Tecnológico y de Estudios Superiores de Monterrey

Abstract— La tarea de inpainting ha sido dominada en años recientes por modelos altamente especializados, entrenados sobre datasets de gran escala y respaldados por arquitecturas complejas que integran mecanismos de atención global y pérdidas adversariales. Si bien estos modelos alcanzan una fidelidad visual notable, su costo computacional —tanto en entrenamiento como en inferencia— limita su uso en escenarios con recursos restringidos. En este trabajo se explora qué tan lejos puede llegar un modelo liviano basado en un autoencoder convolucional inspirado en una U-Net simplificada, entrenado con tan solo 12,000 imágenes de cinco tipos de paisajes. El modelo se optimiza mediante métricas perceptuales sin recurrir a discriminadores ni a módulos de atención. Se evalúa su capacidad para reconstruir regiones faltantes en escenas naturales y se demuestra que, aunque sacrifica parte de la precisión lograda por los métodos de estado del arte, ofrece una inferencia rápida, con baja utilización de memoria y sin necesidad de hardware especializado. Los resultados evidencian tanto el potencial como las limitaciones inherentes a una arquitectura minimalista enfocada en un dominio visual específico.

Palabras clave—aprendizaje profundo, inteligencia artificial, autoencoder, inpainting, reconstrucción, denoiser, LPIPS,

I. INTRODUCCIÓN

LOS autoencoders convolucionales constituyen una clase de redes neuronales profundas capaces de extraer las características esenciales de una imagen y codificarlas en una representación de menor dimensionalidad, conocida como espacio latente, preservando al mismo tiempo su estructura espacial. Esta capacidad permite reconstruir aproximaciones razonables de la entrada sin necesidad de memorizarla explícitamente, lo que los vuelve útiles en tareas como compresión, denoising o reconstrucción básica.

Por su parte, el inpainting busca rellenar regiones faltantes en una imagen aprovechando la información contextual disponible. Los modelos de deep learning empleados en esta tarea aprenden patrones visuales que les permiten generar contenido plausible y coherente con el entorno local y global de la imagen.

La combinación de ambas ideas sugiere que un autoencoder puede servir como punto de partida para un sistema de inpainting, pues comparte el objetivo principal de reconstruir información visual. No obstante, la naturaleza compresiva del autoencoder introduce inevitablemente una pérdida de detalles

finos, lo cual limita su capacidad para reproducir texturas complejas, variaciones de iluminación o estructuras repetitivas. Estas limitaciones motivan la exploración de técnicas perceptuales o arquitecturas más trabajadas para mejorar la calidad de la reconstrucción sin incrementar de forma sustancial el costo computacional.

I.i. Estado del Arte

Los principales modelos que también se basan en autoencoders utilizan la capacidad compresiva para poder aprender las características más importantes de una imagen mientras reducen la dimensionalidad para volverse más eficientes. Comparten el análisis de estructuras generales y específicas de cada imagen para poder aprender a generalizar a gran variedad de escenarios o situaciones, con configuraciones variables de resolución, iluminación, entre otros. Su enorme capacidad adaptativa es lo que los vuelve tan versátiles y eficaces, pero al mismo tiempo es lo que los vuelve cada vez más pesados.

Hay 3 principales arquitecturas que se basan en el autoencoder. Los transformers, que obtienen visión global mediante auto-atención; los diffusers, que usan muchos pasos de deformación y restauración y, las GANs, que se evalúan a sí mismos para aprender muchos más detalles.

I.i.1. Transformers

Los modelos basados en transformers han demostrado un rendimiento sobresaliente en tareas de inpainting gracias a su capacidad para modelar relaciones de largo alcance. A diferencia de las redes convolucionales, estos modelos no dependen de filtros locales, sino que dividen la imagen en parches y procesan cada uno mediante módulos de auto-atención multi-cabezal. Este mecanismo les permite integrar información contextual global desde cualquier región de la imagen y reconstruir los parches enmascarados utilizando relaciones semánticas aprendidas entre todos los parches visibles.

Al operar sobre parches, logran generar reconstrucciones coherentes incluso cuando grandes áreas han sido ocultadas, superando las desventajas del campo receptivo limitado de las convoluciones. Sin embargo, esta capacidad global tiene a su vez sus problemas: presentan mayores dificultades para capturar coherencia local y continuidad de textura en zonas estructuradas, y su entrenamiento requiere conjuntos de datos sustancialmente más grandes que los modelos convolucionales tradicionales.

Asimismo, el proceso de inferencia suele implicar múltiples pasos de atención y decodificación, lo que incrementa considerablemente el costo computacional y la memoria necesaria, especialmente conforme sube la resolución.

I.i.2. Diffusers

Las redes basadas en diffusion usan un proceso iterativo en el que a la imagen se le va añadiendo cada vez más ruido gaussiano hasta que la señal se vuelve totalmente irreconocible, entonces el modelo debe aprender a poder revertir la deformación y así generar una nueva región condicionada en un proceso de igualmente iterativo. Algunos poseen atención cruzada que le permite guiar el proceso con condiciones externas a la imagen, como texto, mientras que otros mantienen un sistema de atención multinivel.

Mezcla características de alto y bajo nivel para conseguir resultados más convincentes a la hora de aprender a predecir el ruido para poderlo eliminar, siendo el caso específico del inpainting poder predecir el contenido detrás de la máscara durante el muestreo para revertirlo en algo coherente de acuerdo al contexto obtenido de la señal inicial.

E salto de conexiones que hereda al basarse en U-Net le da el poder de reconstruir los detalles más finos tanto del contexto global como de estructuras locales, con la contra de que un proceso puramente de difusión es sumamente complicado computacionalmente, ya que además de requerir miles de pasos de muestreo, las redes por sí mismas se construyen de gran profundidad para permitir tal proceso. Tanto para entrenar como para inferir, la naturaleza iterativa de la arquitectura hace que además de lento, requiera de hardware especializado de gran capacidad para poderse ejecutar. Aunque las versiones más recientes mezclan la difusión sobre un autoencoder variacional externo para poder hacer uso de su espacio latente, esto suma entonces problemas de complejidad al tener que entrenar y/o usar un modelo completo por separado.

I.i.3. GAN

Las redes generativas adversariales, o GAN por sus siglas en inglés, eran las dominantes en el ámbito generativo antes de que surgieran las arquitecturas anteriormente detalladas, ampliamente utilizadas, además de para inpainting, en tareas de super resolución y de síntesis. Aunque su popularidad haya decaído, no se han visto desplazadas por completo en situaciones donde la velocidad de inferencia es prioridad, por ejemplo.

Se componen de dos redes que entrenan en oposición mediante un juego minimax: un generador, que crea contenido según como se haya condicionado, y un discriminador, que aprende una función de decisión para determinar si lo que recibe es un objeto real o algo producido por el generado. Entonces en el proceso de entrenamiento, el generador debe aprender a crear contenido cada vez más verosímil para pasar al discriminador mientras este debe aprender a discernir de mejor manera lo que es real de lo que no.

Para el contexto del inpainting, el discriminador suele configurar para únicamente analizar la zona reconstruida por el generador tras la máscara y debe indicarle a este qué tan poco convincente resulta su salida, aunque también se puede implementar para examinar la imagen completa. Esta capacidad de retroalimentación es lo que lo llevó a ser lo más utilizado por

todo ese tiempo, pues al momento de la inferencia, hay un único camino lineal para poder entregar la imagen solicitada, y su arquitectura es flexible a utilizar cualquier base (U-Net, ResNet, encoder-decoder, etc.) mientras se respete la dinámica de generador contra discriminador.

La principal desventaja de este tipo de redes es precisamente el entrenamiento. Aunque no requieran de datasets especialmente etensos, en comparación con un transformer, ni de propagación hacia adelante tardada, hay que construir dos redes independientes con sus propios parámetros, funciones de pérdida e interacciones para poder permitir que uno aprenda del otro, siendo muy sensible a los hiperparámetros. Como el objetivo principal es que en general el generador sea capaz de anteponerse al discriminador, cuando el generador aprende a engañar al discriminador con un mismo mapa en lugar de generalizar soluciones, se produce el denominado “colapso de modo” donde el entrenamiento se estanca.

I.ii. Trabajo relacionado

I.ii.1 Context encoders (Pathak et al., 2016)

En 2016, Pathak et al. [1] introdujeron el concepto de Context Encoders, una arquitectura basada en autoencoders capaz de realizar image inpainting aprovechando tanto el contexto global como la información local circundante a la región faltante. Su propuesta surgió como respuesta a las limitaciones de los métodos predominantes en ese momento, los cuales solían recurrir a estrategias de copiar y pegar parches similares desde regiones visibles, producir reconstrucciones de baja resolución al promediar los valores, o requerir correcciones manuales posteriores para lograr coherencia estructural y semántica.

La clave de su enfoque fue la cuidadosa selección de funciones de pérdida, incluyendo una pérdida de reconstrucción (L2) aplicada a la zona enmascarada para forzar la generación de estructuras, y un discriminador mediante GAN que permita el uso de pérdida adversaria, la cual fuerza la generación de parches plausibles y realistas.

Un discriminador enfocado únicamente en la región reconstruida inclinó al modelo a que las texturas fueran realistas y las estructuras con sentido respecto al resto de la imagen. Esto permitió obtener resultados superiores a los métodos tradicionales.

La arquitectura fue nombrada así al tener la capacidad de reunir el contexto global de la imagen así como el local del área a reparar para poder inferir el contenido faltante sin recurrir a parches basados en sus alrededores. Sentó las bases para el avance del Inpainting mediante aprendizaje profundo.

I.ii.2. Partial convolutions (Liu et al., 2018)

Hasta entonces, los métodos de inpainting basados en deep learning empleaban convoluciones estándar, ya fuera en redes completamente convolucionales o combinadas con capas densas. Aunque las arquitecturas variaban, todas compartían una limitación clave: las convoluciones procesaban por igual tanto los píxeles válidos como los píxeles enmascarados, lo que generaba artefactos como derramado de color, difuminación o aliasing de estructuras.

En su trabajo, Liu et al. [2] centraron su contribución no en una arquitectura nueva, sino en el diseño de una nueva operación convolucional, denominada Partial Convolution (PConv). En

una convolución parcial, los valores enmascarados se excluyen por completo del cómputo, de manera que el resultado se calcula únicamente a partir de los píxeles válidos dentro del kernel. Además, cada operación produce no solo un mapa de características, sino también una máscara actualizada, que indica qué regiones han sido ya “rellenadas” por información confiable.

Formalmente, una convolución parcial se considera válida cuando al menos un píxel real dentro del filtro contribuye al cálculo. Conforme la red avanza, la máscara se va reduciendo hasta que eventualmente todos los filtros reciben valores reales, lo que permite que capas profundas de la red capturen contexto global incluso en zonas originalmente vacías.

Para implementar esta nueva operación, Liu et al. utilizaron una arquitectura basada en U-Net, aprovechando sus saltos de conexión para combinar tanto las características como las máscaras generadas en cada nivel. Este enfoque permitió mejorar la estabilidad en regiones grandes, evitar la propagación de información inválida y producir reconstrucciones más limpias que las obtenidas con convoluciones estándar.

I.ii.3. Large Mask Inpainting (Suvorov et al., 2022)

En 2022, en pleno auge de los métodos generativos modernos, Suvorov et al. [3] introdujeron LaMa, un modelo que marcó un avance significativo en image inpainting gracias a su capacidad de producir reconstrucciones de alta fidelidad y notable robustez incluso en resoluciones muy superiores a las utilizadas durante el entrenamiento.

El enfoque de LaMa se centra en controlar y ampliar el campo receptivo efectivo de la red. Los autores argumentan que, para reconstruir huecos grandes, el modelo debe tener acceso al contexto global de la imagen desde etapas tempranas, de modo que pueda entender relaciones semánticas de largo alcance. Esto los llevó a diseñar una arquitectura que extiende el campo receptivo tanto en sus bloques convolucionales como en su función de pérdida.

LaMa toma inspiración de la Convolución de Fourier, una operación realizada en el dominio de la frecuencia que permite aplicar kernels muy amplios sin el costo computacional de las convoluciones espaciales. Para integrarla adecuadamente, los autores diseñaron el Fast Fourier Convolution (FFC): un bloque que separa la entrada en dos ramas: una rama local, basada en convoluciones estándar, y una rama global, que aplica una Transformada Rápida de Fourier (FFT), procesa la información en el dominio de la frecuencia y luego la regresa al dominio espacial. De esta manera se obtiene simultáneamente coherencia local con consistencia global.

Además del módulo arquitectónico, LaMa incorpora una función de pérdida diseñada específicamente para este objetivo. Su HRF loss es una pérdida perceptual que utiliza un modelo preentrenado con un campo receptivo particularmente amplio. La idea es comparar las activaciones del modelo objetivo con las del modelo maestro para asegurar que la reconstrucción mantenga consistencia semántica a nivel global. Esta pérdida se suma a otras como la reconstrucción, la perceptual tradicional y la adversaria, componiendo una función de pérdida integral que guía el entrenamiento.

Su arquitectura final era una GAN basada en U-Net donde las FFC se concentran en el cuello de botella mientras el encoder-

decoder realiza funciones habituales. Así permite generar resultados de alta calidad con el apoyo del discriminador y la amplia función de pérdida.

Aunque modelos basados en Transformers o Difusión presentan resultados superiores a LaMa, su propuesta representó uno de los mayores pasos hacia un inpainting robusto y adaptable.

II. METODOLOGÍA

Una arquitectura de autoencoder basada en U-Net fue empleada para resolver la tarea, inspirada en las observaciones de los desarrolladores de LaMa y manteniendo la premisa de simplicidad en el diseño. En consecuencia, la función de pérdida se compone únicamente de dos términos, se utilizan convoluciones regulares en todas las capas y las métricas de evaluación se mantienen deliberadamente simples.

II.i. Arquitectura

Como red basada en U-Net [4], la estructura encoder-decoder mantiene una simetría exacta, tratándose de una red completamente convolucional con una profundidad de tres bloques por etapa y un cuello de botella compuesto por dos capas. Todas las capas emplean activaciones ReLU, una elección común por su eficiencia computacional y por evitar la saturación del gradiente durante el entrenamiento.

II.i.1. Primera parte - Encoder

La primera capa oculta del modelo es una convolución con stride 2 que, junto con un padding “same”, reduce a la mitad la resolución espacial. El kernel utilizado es de 7×7 . Suvorov et al. también señalan que una manera eficaz de ampliar la visión global en etapas tempranas es aumentar el campo receptivo mediante kernels grandes. Para limitar el costo computacional, se optó por un único kernel ampliado de 7×7 en lugar de varias convoluciones secuenciales de 5×5 . La red inicia con 32 filtros, suficientes para capturar contornos y estructuras de bajo nivel.

II.i.2. Segunda parte - Encoder

La segunda y tercera capa del modelo tienen como única función seguir reduciendo a la mitad la resolución mientras duplican el número de filtros, con 64 y 128 filtros respectivamente, para capturar estructuras cada vez más amplias.

II.i.3. Tercera parte - Encoder

La cuarta capa del modelo ya no reduce la resolución, pues es importante no comprimir en exceso el espacio latente para evitar la pérdida de detalles. En esta etapa se duplican nuevamente los filtros hasta 256 para procesar contornos más complejos, y preparar la representación que ingresa al cuello de botella.

II.i.4. Cuello de botella

El espacio latente está compuesto por dos capas consecutivas de 512 filtros que mantienen la misma resolución. En esta etapa se integran convoluciones dilatadas [5], introducidas por Yu et al. como una forma de ampliar el campo receptivo sin aumentar el número de parámetros. Por ejemplo, un kernel 3×3 con

dilatación 2 equivale, en campo receptivo, a un kernel 5×5 . Esto permite que el modelo combine información global y local sin sacrificar detalle.

La fórmula que describe el campo receptivo efectivo de una convolución dilatada es:

$$RF = (K - 1) * d + 1$$

donde:

- K es el tamaño del kernel
- d es el ratio de dilatación

II.i.4.a Campo receptivo acumulado

Capa	Kernel	Stride	Dilation	Jump	RF
Entrada	7	2	1	2	7
Enc1	3	2	1	4	11
Enc2	3	2	1	8	19
Enc3	3	1	1	8	35
Cuello1	3	1	2	8	67
Cuello2	3	1	2	8	99

Tabla 1. Cálculo del campo receptivo por capa

II.i.5. Decoder

En simetría con el encoder, el decoder reconstruye la imagen aumentando progresivamente la resolución. Para ello, cada bloque que requiere ampliación aplica upsampling bilineal antes de la convolución. Además, siguiendo el diseño de U-Net, se emplean saltos de conexión entre cada capa del encoder y su contraparte del decoder, lo que permite recuperar información espacial perdida durante la compresión y ayuda a mejorar la calidad de los detalles reconstruidos.

II.i.6. Normalización

Aunque Batch Normalization es habitual en tareas de visión, su uso puede introducir inconsistencias en tareas generativas debido a que mezcla estadísticas entre muestras heterogéneas dentro de un batch. Para evitarlo, se emplea Instance Normalization [6], propuesta inicialmente para transferencia de estilo. Esta técnica normaliza cada imagen de manera independiente, evitando la contaminación entre escenas distintas y conservando mejor los detalles específicos de cada paisaje. Se aplica después de cada convolución, excepto en la capa de salida.

II.ii. Función de pérdida

La función de pérdida utilizada debe cumplir dos objetivos principales: lograr una alta calidad visual en la reconstrucción (“que se vea bien”) y mantener coherencia semántica (“que el parche tenga sentido”). En trabajos recientes, esto suele abordarse con funciones de pérdida compuestas, que combinan diversos términos ponderados para evaluar distintos aspectos de la imagen. Sin embargo, siguiendo la premisa de simplicidad, este modelo emplea únicamente dos términos complementarios.

II.ii.1. Pérdida de reconstrucción

La pérdida de reconstrucción cuantifica cuán precisa es la imagen generada con respecto a la original, comparándolas píxel a píxel. Es el principal impulsor de la fidelidad estructural

y textural del modelo, por lo que recibe el mayor peso en la función total. Las pérdidas más comunes son el Error Cuadrático Medio (L2/MSE) y el Error Absoluto Medio (L1/MAE). Sin embargo, ambas presentan dificultades para tareas de inpainting.

MSE es altamente sensible a outliers: un solo píxel con gran desviación puede aumentar notablemente el error. MAE, en contraste, penaliza todos los errores de manera uniforme, lo que hace que sus gradientes sean menos informativos y puede ralentizar el aprendizaje.

Para resolver estas limitaciones, Charbonnier et al. introdujeron una variante suave y robusta originalmente diseñada para preservar bordes en procesamiento de imágenes. Años después se popularizó como pérdida de Charbonnier [7] o pérdida de Charbonnier generalizada, definida como:

$$p(x) = \sqrt{x^2 + \epsilon^2}$$

Esta función combina las ventajas de L1 y L2: para errores grandes actúa de forma similar a MSE, mientras que cerca de cero se comporta como una versión suavizada de MAE. Esto la hace robusta tanto a outliers como a errores pequeños. Además, gracias a la constante ϵ , es derivable incluso en $x = 0$.

Por estos motivos, la pérdida de Charbonnier fue seleccionada como término principal de reconstrucción.

II.ii.2. Pérdida perceptual

Las pérdidas de reconstrucción no pueden emplearse como única medida del error por dos razones: no exigen que el parche generado se integre con el resto de la imagen y limitan la libertad del modelo para decidir qué contenido debería existir en la región faltante. Por ello, aun si la predicción aislada se ve correcta, al observar la imagen completa puede percibirse como un parche sobrepuesto.

Este problema se aborda mediante las pérdidas perceptuales, diseñadas para medir la similitud de forma más cercana a la percepción humana en lugar de comparaciones estrictas píxel a píxel. Estas pérdidas permiten cierta tolerancia a pequeños errores locales a cambio de mantener la coherencia global del contenido y de la estructura visual.

Learned Perceptual Image Patch Similarity, o LPIPS [8], es una de las pérdidas perceptuales más empleadas. Utiliza redes neuronales previamente entrenadas (típicamente AlexNet o VGG16) para extraer mapas de activación de ambas imágenes y calcular la distancia entre ellos: valores más cercanos a 0 indican mayor similitud perceptual. Suele utilizarse VGG16 durante el entrenamiento de modelos profundos, mientras que AlexNet se emplea con más frecuencia como métrica de evaluación por su menor costo computacional.

LPIPS complementa de forma efectiva la pérdida de reconstrucción, pues orienta al modelo hacia reconstrucciones coherentes y visualmente integradas. Dado que su función es de soporte, su peso relativo es menor, aunque no despreciable, en comparación con la pérdida principal.

En este trabajo se utilizó VGG16 gracias a la disponibilidad de hardware suficiente para soportarla, aunque existen alternativas más ligeras que pueden emplearse con ajustes apropiados.

II.iii. Preparación de los datos

II.iii.1 Dataset ([Enlace](#))

El conjunto de datos empleado está conformado por imágenes pertenecientes a cinco tipos de paisaje: bosque, desierto, costa, montaña y glaciar. El dataset fue originalmente diseñado para una tarea de clasificación, por lo que su autor se encargó de mantener todas las clases perfectamente balanceadas, facilitando así el entrenamiento.

En total contiene 12,000 imágenes de 174x290, divididas equitativamente entre las cinco clases. Los datos ya vienen separados por tipo de conjunto:

- Entrenamiento: 2,000 imágenes por clase
- Validación: 300 imágenes por clase
- Pruebas: 100 imágenes por clase

Las imágenes no se limitan a fotografías directas de paisajes. También incluyen dibujos, imágenes extraídas de otros medios, fotografías con marcas de agua o texto y paisajes acompañados de elementos ajenos como personas u obras humanas. Esto introduce variabilidad en la distribución visual y eleva la dificultad del problema de inpainting.

II.iii.2 Preprocesamiento

Para mantener los costos computacionales manejables en una red relativamente profunda, no se emplean técnicas extensivas de preprocesamiento como aumento de datos o depuración manual. Únicamente se realiza la normalización necesaria y la generación automática de las máscaras.

II.iii.3 Estandarización

Las imágenes se recortan al múltiplo de 8 más cercano, ya que la arquitectura divide la resolución tres veces por la mitad y requiere que todas las operaciones mantengan dimensiones enteras. Esto produce un tamaño de entrada de 160×272 píxeles. Además, los valores de color se normalizan al intervalo [0, 1] en formato float32, lo que permite que todas las operaciones se realicen directamente en este dominio sin conversiones adicionales.

II.iii.4 Políticas para la generación de máscaras

Los desarrolladores de LaMa demostraron que las máscaras más efectivas para entrenar modelos de inpainting son agresivas, llegando a cubrir hasta el 50 % de la imagen y utilizando formas variadas como rectángulos, círculos o trazos curvos tipo pincel. Sin embargo, dado que este modelo no cuenta con los bloques FFC que proporcionan visión global, se mantiene la misma lógica de formas pero con tamaños más moderados, acordes al alcance del campo receptivo disponible. Las máscaras se generan de manera dinámica durante la creación de cada batch, de modo que la red ve una máscara distinta para la misma imagen en cada iteración. Con un máximo de 10 figuras por máscara y un tamaño máximo de 50 píxeles por figura, la función selecciona aleatoriamente: su posición, su grosor y si dibuja un rectángulo, un círculo o una línea redondeada.



Figura 1. Ejemplo de máscara generada

II.iv. Entrenamiento

El modelo se implementó en TensorFlow utilizando Keras, empleando únicamente componentes nativos del framework salvo la función de pérdida perceptual. La implementación original de LPIPS fue desarrollada para PyTorch, y aunque existe una adaptación oficial para TensorFlow, esta se encuentra desactualizada y no es compatible con las versiones actuales. Por ello, en este trabajo se utilizó la adaptación publicada por Image X Institute [9] en 2024.

II.iv.1. Hiperparámetros

La ausencia de técnicas de aumento de datos o transformaciones dinámicas permitió utilizar un tamaño de batch de 32, un valor estándar incluso en hardware no especializado. Como procesadores y GPUs suelen estar optimizados para tamaños potencia de dos, un batch de 32 ofrece un buen equilibrio entre rendimiento y estabilidad. En equipos más limitados puede emplearse un batch de 16 sin afectar significativamente el comportamiento del modelo.

El entrenamiento se llevó a cabo durante 200 épocas, dado que se trata de una tarea compleja y la pérdida perceptual (LPIPS) requiere suficiente tiempo para refinar detalle. Aun así, resultados razonablemente coherentes comienzan a aparecer alrededor de las 60 épocas.

Respecto a la función de pérdida, la imagen válida se evalúa mediante Charbonnier sin modificar su ponderación, mientras que las regiones enmascaradas reciben un peso doble, seguido de una normalización que evita que la dimensionalidad influya en la magnitud del término. A esto se suma LPIPS con un factor de 0.8, aplicada sobre la imagen completa, incluyendo regiones válidas y enmascaradas.

II.iv.2. Optimizador

Como optimizador se utilizó Adam, un estándar ampliamente adoptado gracias a su combinación de Momentum y RMSProp, que permite una convergencia rápida en las primeras etapas del entrenamiento y ajusta dinámicamente la tasa de aprendizaje para cada parámetro. Su estabilidad lo convierte en una opción robusta para una gran variedad de arquitecturas sin requerir ajustes específicos. En este modelo, Adam se empleó sin fijar explícitamente una tasa de aprendizaje constante.

II.iv.3. Programador de tasa de aprendizaje

Para favorecer la convergencia, resulta útil emplear una tasa de aprendizaje alta en las primeras épocas—cuando los cambios son drásticos—y reducirla conforme el modelo requiere ajustes finos. Con el programador ReduceLROnPlateau, la tasa se adapta automáticamente cuando el entrenamiento entra en zonas de estancamiento (“mesetas”), que suelen corresponder a mínimos locales o soluciones parciales.

La tasa inicial se fijó en 2×10^{-3} , elevada para impulsar las mejoras iniciales, junto con un factor de decaimiento de 0.5. La paciencia de 5 épocas concede margen para que el modelo

explore alternativas sin prolongar innecesariamente el entrenamiento cuando el avance se detiene. Este procedimiento reduce la tasa hasta alcanzar el mínimo establecido de 1×10^{-4} , aún suficiente para actualizaciones pequeñas, pero no despreciables.

II.iv.4. Flujo de entrenamiento

Antes de cada época, el conjunto de datos completo se baraja aleatoriamente y se aplican la estandarización y la generación de máscaras, a partir de las cuales se construyen los batches.

Para las máscaras, en cada época se genera una nueva por imagen, duplicada en los tres canales y aplicada directamente en lugar de agregarse como canal adicional. Esta máscara RGB se concatena a la imagen objetivo para ser procesada por la función de pérdida personalizada, que también gestiona la ponderación diferenciada de las regiones válidas y enmascaradas.

Este procedimiento evita que el modelo vea la misma combinación exacta de imagen + máscara o el mismo orden de los datos, reduciendo la posibilidad de que encuentre atajos en la solución.

II.iv.5. Otros monitoreos

Tras cada época se evalúa la pérdida sobre el conjunto de validación, lo que permite activar dos mecanismos adicionales además del programador de tasa de aprendizaje:

- Punto de guardado (checkpoint): cada vez que la pérdida de validación disminuye, se guarda automáticamente una nueva versión del modelo para evitar pérdida de progreso frente a imprevistos.
- Paro temprano (early stop): con una paciencia de 20 épocas. Si la pérdida de validación no mejora durante ese periodo, pese a que el programador ya se haya activado varias veces, el entrenamiento finaliza y se restaura la mejor versión del modelo.

Ambos mecanismos previenen el sobreajuste, evitan entrenamientos prolongados innecesarios y protegen el avance del modelo ante fallos externos.

II.v. Evaluación

Como un modelo de inpainting es una tarea inherentemente ambigua, pues no existe una única manera “correcta” de rellenar un hueco, no se dispone de métricas clásicas de aprendizaje supervisado que establezcan una verdad absoluta. Por ello, la evaluación se apoya en métricas pixel a pixel, perceptuales, y basadas en modelos preentrenados, cada una capturando un aspecto distinto de la calidad de la reconstrucción.

Además de la inspección visual humana, la tradición en procesamiento de imágenes sugiere el uso de métricas como PSNR, SSIM y LPIPS, las cuales en conjunto permiten juzgar fidelidad local, coherencia estructural y consistencia perceptual del resultado. Horé et al. (2010) [10] comparan precisamente PSNR y SSIM al evaluar calidad de imagen, lo que sustenta su uso combinado.

II.v.1. SSIM

El índice de similitud estructural (SSIM) es una métrica perceptual ampliamente aceptada para evaluar calidad de

imágenes reconstruidas. En lugar de comparar valores de píxeles directamente, SSIM evalúa cómo se preservan tres componentes fundamentales de la percepción visual:

- Luminancia: compara el brillo global de ambas imágenes.
- Contraste: mide la variación relativa entre intensidades.
- Estructura: evalúa la correlación local de patrones, bordes y texturas.

El resultado se normaliza generalmente al intervalo $[0, 1]$, donde valores más altos indican una mayor similitud estructural. SSIM es particularmente útil para inpainting porque penaliza distorsiones que alteran la percepción global de la escena aún si el error numérico es bajo.

Dado su enfoque estructural, SSIM servirá como contraparte perceptual de LPIPS, enfocándose más en la coherencia local que en la similitud de alto nivel. Su uso complementa adecuadamente al resto de métricas, ya que detecta distorsiones que la pérdida perceptual puede pasar por alto.

II.v.2 PSNR

La relación máxima señal-ruido (PSNR) es una métrica basada en el error cuadrático medio entre la imagen original y la reconstruida. Su propósito principal es cuantificar la distorsión introducida durante la reconstrucción y se expresa en decibelios (dB).

Aunque PSNR proviene de contextos como compresión con pérdida y restauración, también es útil en inpainting porque mide qué tanto se alteraron los valores originales y cuánto ruido se ha introducido.

En general:

- >40 dB: excelente calidad
- $30-40$ dB: buena calidad
- $20-30$ dB: calidad aceptable
- < 20 dB: calidad pobre

Como PSNR es insensible a la percepción humana, puede otorgar valores altos a imágenes que se ven artificiales, lo que refuerza la necesidad de combinarlo con métricas perceptuales. Aquí, PSNR permite cuantificar el grado de deformación residual que LPIPS y SSIM podrían no penalizar.

II.v.3. LPIPS

LPIPS se utilizará tanto como pérdida durante el entrenamiento como métrica de evaluación independiente. A diferencia de SSIM, LPIPS no mide directamente luminancia o contraste, sino la distancia perceptual entre imágenes utilizando modelos de aprendizaje profundo como se mencionaba anteriormente.

Esta métrica captura características de alto nivel como la coherencia semántica, la preservación de texturas globales, la naturalidad percibida y los patrones identificados en objetos reales. Valores más bajos de LPIPS indican una mayor similitud perceptual.

Debido a que LPIPS evalúa imágenes en un espacio de características de redes profundas, logra identificar deformaciones o detalles que un humano detectaría fácilmente pero que SSIM o PSNR podrían pasar por alto. Esto lo convierte en el complemento ideal para las otras métricas, cerrando la brecha entre evaluación numérica y evaluación humana.

III. RESULTADOS



Figura 2. Imagen original junto a su reconstrucción.

Durante la exposición de resultados, las comparativas habitualmente serán realizadas frente a una versión preliminar del modelo, el cual sólo posee 20 épocas de entrenamiento y elimina tanto las convoluciones dilatadas como la segunda capa del cuello de botella, con un cambio importante que es el uso de convoluciones transpuestas en el decoder. Este modelo base sirve como referencia mínima para evaluar el impacto real de las modificaciones arquitectónicas propuestas y del entrenamiento completo. A partir de esta línea base, se presentan los resultados cualitativos y cuantitativos obtenidos por la versión final del modelo.

II.i. Evolución del entrenamiento

El modelo se entrenó durante 200 épocas, distribuidas en dos sesiones consecutivas de 100 épocas, manteniendo la continuidad del programador de tasa de aprendizaje para evitar reinicios de su estado interno.

A lo largo del entrenamiento, las curvas de pérdida de entrenamiento y validación se mantuvieron cercanas y estables, con la pérdida de validación siendo ligeramente menor en la mayoría de los casos. Esta relación indica que el modelo no experimentó problemas de sobreajuste ni subajuste, ya que fue capaz de generalizar adecuadamente sin memorizar patrones específicos del conjunto de entrenamiento.

Durante las primeras 80 épocas, la curva muestra una inestabilidad característica de tareas de reconstrucción con componentes perceptuales: aparecen picos y valles pronunciados en la pérdida. Estas oscilaciones representan fases donde el modelo encuentra soluciones parciales que no logran consolidarse sin la intervención del programador, el cual reduce la tasa de aprendizaje en zonas de estancamiento. Sin este mecanismo, la optimización podría haberse detenido prematuramente.

Tras esta fase inicial, las oscilaciones comienzan a disminuir gradualmente. A partir de aproximadamente la época 90, las pérdidas muestran un comportamiento mucho más estable, con un descenso suave y consistente hasta su convergencia. En la última etapa del entrenamiento, el modelo ajusta detalles finos en la reconstrucción, beneficiándose del ritmo reducido de aprendizaje impuesto por el programador.



Figura 3. Curvas de pérdida durante el entrenamiento

III.ii. Resultados cuantitativos

Modelo base		Modelo final	
Métrica	Resultado	Métrica	Resultado
PSNR	23.6 dB	PSNR	24.5 dB
SSIM	0.73	SSIM	0.79
LPIPS	0.18	LPIPS	0.16

Tabla 2. Promedio de métricas en 5 batches (160 imágenes)

Para calcular las métricas, se utilizaron tanto el modelo base como el modelo final para obtener predicciones sobre máscaras generadas aleatoriamente a lo largo de 5 batches del conjunto de pruebas, es decir, imágenes que no fueron utilizadas durante ninguna parte del entrenamiento. Se empleó un iterador que garantizó que ninguna imagen o máscara se repitiera en el proceso de evaluación, asegurando así la validez estadística del promedio reportado.

En primera instancia, los resultados del modelo preliminar ya se ubican dentro de la categoría de “Decente”, alcanzando un PSNR de 23.6 dB y una similitud estructural de 0.73 en SSIM. La métrica perceptual adicional reporta igualmente una diferencia humana muy pequeña. Esto confirma la solidez de la arquitectura incluso sin los elementos añadidos posteriormente, y además respalda la afirmación de que los mayores avances y la obtención de resultados aceptables ocurren durante las primeras ~60 épocas presentada con anterioridad.

Con respecto al modelo final, en concordancia con la idea de que las épocas adicionales se destinan principalmente a refinar detalles de menor escala, la mejora cuantitativa es relativamente modesta, aunque consistente. Si bien el incremento no es lo suficientemente amplio como para cambiar de categoría perceptual, sí representa una mejora real y observable, posicionándose cerca del umbral de la categoría “Buena”.

En resumen, las métricas cuantitativas sugieren que el entrenamiento prolongado tiene un efecto claro en la refinación de texturas y detalles secundarios, mientras que los avances más significativos para la reconstrucción global de la imagen ocurren en una fracción considerablemente menor del total de épocas.

III.ii. Resultados cualitativos

Si bien las métricas cuantitativas ofrecen una aproximación sólida al desempeño del modelo y correlacionan

adecuadamente con la percepción humana, no permiten describir con totalidad la naturaleza de los errores ni la calidad visual del resultado final. Por ello, se presenta un análisis cualitativo complementario para ambos modelos.



Figura 4. Imagen original junto a su reconstrucción, modelo preliminar

En la Figura 4 se observa que el modelo preliminar logra reconstruir correctamente las estructuras globales de la imagen, preservando siluetas y colores dominantes. Sin embargo, exhibe un tipo particular de aliasing: el artefacto en forma de tablero de ajedrez, o checkerboard artifact.

Este patrón surge típicamente cuando se emplean convoluciones transpuestas y existe una incompatibilidad entre el tamaño del kernel y el desplazamiento (stride), provocando un muestreo irregular que distribuye el contenido de manera no uniforme. Como resultado, aparecen patrones cuadrículados claramente visibles, especialmente en regiones lisas o de textura suave.

La presencia insistente de este fenómeno motivó el rediseño del decoder, reemplazando las convoluciones transpuestas por capas de upsampling seguidas de convoluciones regulares, con el fin de eliminar esta fuente de aliasing estructural.



Figura 5. Imagen junto a su reconstrucción, modelo final

En contraste, el modelo final, apreciable en la figura 5, muestra una reducción notable del checkerboard artifact, junto con una disminución general en la difuminación de detalles. Este avance se debe tanto a la modificación arquitectónica del decoder como al mayor peso otorgado a la métrica perceptual LPIPS durante el entrenamiento, lo que penaliza más fuertemente las desviaciones texturales.

Como consecuencia, las texturas finas se vuelven más reconocibles y la coherencia local aumenta. No obstante, el resultado introduce un nuevo tipo de aliasing: un patrón leve en forma de zigzag. Este patrón es menos intrusivo que la cuadrícula original, pero refleja la compensación típica entre suavidad, nitidez y estabilidad perceptual que ocurre al ajustar la combinación entre pérdidas de reconstrucción y pérdidas perceptuales.

III.iii. Análisis de errores

Pese a los esfuerzos por ajustar la arquitectura sin recurrir a mecanismos más avanzados pero también más eficaces, el principal problema del modelo continúa siendo el aliasing. En esta configuración, donde únicamente se emplean dos términos de pérdida (Charbonnier y LPIPS), no existe un componente adicional capaz de supervisar explícitamente las texturas de alta frecuencia, como podría hacerlo una pérdida adversaria o un término especializado en bordes. Debido a ello, el modelo sólo puede reducir los artefactos tanto como se lo permita el tiempo de entrenamiento.

Dentro de esta limitación, se presenta un compromiso inherente entre las métricas utilizadas. Dar mayor peso a Charbonnier mejora notablemente PSNR y SSIM al favorecer reconstrucciones estructuralmente más precisas, pero a costa de producir imágenes más difuminadas. Por el contrario, incrementar la ponderación de LPIPS tiende a recuperar mejor las texturas perceptuales, pero frecuentemente introduce un patrón característico de aliasing. La configuración final representa un punto medio que resulta visualmente más equilibrado, aunque se refleja en una mejora cuantitativa poco significativa frente al modelo preliminar.

Como era de esperarse por la naturaleza del aliasing, el modelo presenta mayores dificultades en escenas donde predominan texturas altamente complejas o caóticas, como paisajes rocosos o vegetación densa. En estos casos, los artefactos en forma de zigzag se vuelven mucho más evidentes, mientras que en escenarios con patrones más homogéneos (desiertos, glaciares o cielos), el modelo logra reconstrucciones más limpias y menos propensas a errores.

Siguiendo la política de las máscaras, la calidad de la reconstrucción también resulta inversamente proporcional al tamaño de la región enmascarada. Incluso en las zonas donde la arquitectura muestra mayor fortaleza, reconstruir áreas extensas provoca una caída notable en nitidez respecto a cuando sólo se requieren pequeñas correcciones locales. Este comportamiento reafirma que las capacidades del modelo dependen fuertemente tanto del contexto circundante como del tipo de textura presente en la imagen.

IV. CONCLUSIONES

En este trabajo se evaluó el desempeño de una arquitectura de reconstrucción de imágenes a partir de regiones enmascaradas, analizando tanto una versión preliminar del modelo como una versión final con ajustes en el decoder, el cuello de botella y el tiempo de entrenamiento. Los resultados permiten establecer varias conclusiones claras.

Primero, incluso con una configuración reducida, el modelo preliminar demostró una capacidad sólida para reconstruir la estructura global y los colores principales de las imágenes. Su rendimiento cuantitativo se situó dentro de un nivel “decente” según rangos habituales de PSNR y SSIM, lo cual confirma que la arquitectura base es funcional y que gran parte del aprendizaje significativo ocurre en las primeras épocas de entrenamiento.

Segundo, los cambios introducidos en la versión final, principalmente la sustitución de las convoluciones transpuestas por un esquema de upsampling seguido de convoluciones

regulares, y la incorporación de las dilataciones, lograron disminuir de forma notable el aliasing tipo checkerboard y mejorar la nitidez local. No obstante, estas mejoras visuales no se reflejaron con la misma magnitud en las métricas cuantitativas, lo cual indica que tales métricas no capturan completamente los artefactos perceptuales más visibles.

Tercero, los resultados cualitativos y el análisis de errores ponen en evidencia la persistencia de patrones de aliasing, ahora con forma de zigzag, especialmente en zonas con texturas de alta frecuencia como vegetación densa o formaciones rocosas. Esto revela la necesidad de adoptar mecanismos adicionales, ya sean pérdidas orientadas a texturas, términos adversarios o arquitecturas con mayor sensibilidad a detalles de alta frecuencia si se desea eliminar estos artefactos sin recurrir a un difuminado excesivo de la imagen.

Finalmente, se concluye que el entrenamiento prolongado y el balanceo de ponderaciones para una función de pérdida tan simple, permiten impulsar una arquitectura simple sin la necesidad de incorporar mecanismos más elaborados. La configuración actual permite un balance entre nitidez, estabilidad y, por sobre todo, costo computacional. De acuerdo a necesidades más específicas y contextos en particular, los resultados pueden beneficiarse de la alteración de cualquiera de estos tres parámetros establecidos. Pero para fines de este trabajo, el rendimiento seguirá limitado por la complejidad del paisaje y la extensión de la máscara.

V. REFERENCIAS

- [1] D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell y A. A. Efros, «Context Encoders: Feature Learning by Inpainting,» CVPR, 2016.
- [2] G. Liu, F. A. Reda, K. J. Shih, T.-C. Wang, A. Tao y B. Catanzaro, «Image Inpainting for Irregular Holes Using Partial Convolutions,» NVIDIA, 2018.
- [3] R. Suvorov, E. Logacheva, A. Mashikhin, A. Remizova, A. Ashukha, A. Silvestrov, N. Kong, H. Goka, K. Park y V. Lempitsky, «Resolution-robust Large Mask Inpainting with Fourier Convolutions,» WACV, Moscow, 2022.
- [4] O. Ronneberger, P. Fischer y T. Brox, «U-Net: Convolutional Networks for Biomedical Image Segmentation,» Computer Science Department and BIOS Centre for Biological Signalling Studies, Freiburg, 2015.
- [5] F. Yu y V. Koltun, «Multi-Scale Context Aggregation by Dilated Convolutions,» ICLR, 2015.
- [6] D. Ulyanov, A. Vedaldi y V. Lempitsky, «Instance Normalization: The Missing Ingredient for Fast Stylization,» 2016.
- [7] P. Charbonnier, L. Blanc-Féraud, G. Aubert y M. Arlaud, «Deterministic Edge-Preserving Regularization in Computed Imaging,» IEEE TRANSACTIONS ON IMAGE PROCESSING, 1997.
- [8] R. Zhang, P. Isola, A. A. Efros, E. Shechtman y O. Wang, «The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,» CVPR, 2018.
- [9] J. Grover y D. E. Waddington, «lpips_torch2tf,» Image X Institute, 15 Julio 2024. [En línea]. Available: https://github.com/Image-X-Institute/lpips_torch2tf.
- [10] A. Horé y D. Ziou, «Image Quality Metrics: PSNR vs. SSIM,» 20th International Conference on Pattern Recognition, Istanbul, 2010.